# 📘 Apunte completo

Curso **Software Seguro**

Resumen de todo el curso explicado en criollo, con ejemplos, analogías y mapa conceptual.
Módulos I a VI · Taxonomía de ataques · Herramientas · Cómo se reporta

USO EDUCATIVO · REPASO PERSONAL

> 🧭 **La idea que ordena TODO el curso**
>
> Casi todas las vulnerabilidades web se resumen en una frase:
>
> **«El sistema confía en algo que viene del lado del usuario sin verificarlo del lado del servidor.»**
>
> El navegador está en manos del atacante: puede cambiar la URL, el cuerpo, los headers, las cookies, los precios, los IDs… todo. Regla de oro: **nunca confíes en el cliente, validá siempre en el servidor.**

## 🟦 Módulo I — Cómo viaja la información (arquitectura y HTTPS)

### 1. Mapeo de arquitectura lógica

```
Usuario → Frontend → [HTTPS/TLS] → Backend → Base de Datos
                                        ↓
Usuario ← Frontend ← [HTTPS/TLS] ← Backend ← (resultado)
```

- **Frontend:** lo que ves. Vive en tu navegador → **no es confiable**.
- **Backend:** el servidor que decide. Acá deben estar los controles.
- **Base de datos:** usuarios, claves (hasheadas), datos.
- **HTTPS/TLS:** el túnel cifrado por donde viajan los datos.

### 2. Laboratorio Ventas — automatizar con Python

El endpoint `/ventas/?id=1` devolvía códigos distintos: **403** (existe pero sin permiso) y **404** (no existe). Se recorrieron los IDs con `requests`. Cuando un ID editable muestra cosas distintas según el valor → puerta de un **IDOR / control de acceso roto**.

### 3. TLS Handshake — el "apretón de manos" de HTTPS

1. El navegador pide hablar seguro.
2. El servidor manda su **certificado digital** (su "DNI").
3. El navegador verifica que sea válido y del dominio correcto.

4. Acuerdan una **clave secreta** y a partir de ahí todo va cifrado.

*Analogía:* dos personas que se muestran el documento y acuerdan un idioma propio antes de contarse un secreto.

### 4. Instalación de proxy (Burp Suite)

```
Navegador → [ Burp Suite 127.0.0.1:8080 ] → Servidor
```

Un proxy de interceptación se mete en el medio para **ver y modificar** peticiones (ej. `GET /api/gastos/`, `POST /api/gastos/1/editar/`). Es la herramienta #1 del pentester web.

## 🟩 Módulo II — Reconocimiento (mirar antes de tocar)

### 1. Puertos y servicios esenciales

*Analogía:* la IP es el edificio; los **puertos** son las puertas.

| Puerto | Servicio | ¿Seguro? |
|---|---|---|
| 20/21 | FTP (archivos) | ❌ no cifra |
| 22 | SSH (acceso remoto) | ✅ cifrado |
| 23 | Telnet | ❌ obsoleto |
| 80 | HTTP | ❌ sin cifrar |
| 443 | HTTPS | ✅ cifrado |
| 3306 | MySQL (base de datos) | ⚠️ no debería estar expuesto |

### 2. Escaneo con Nmap

**Nmap** dice qué puertos/servicios están abiertos. **Regla ética:** solo con autorización — se usó un programa de **Bug Bounty** (GitHub) revisando el *scope*.

```
nmap -sV app.ejemplo.com   # -sV detecta versiones de los servicios
```

### 3. Reconocimiento de subdominios

En `mail.google.com`, `mail` es el subdominio. Cada subdominio (`app.`, `api.`, `dev.`, `staging.`) puede ser una app distinta, y a veces una está mal protegida.

## 🟧 Módulo III — Interceptar y manipular peticiones (Burp a fondo)

### 1. Man-in-the-Middle local

Se configura Burp como proxy y se instala su **certificado CA** para ver también el tráfico **HTTPS**. *Analogía:* abrir el sobre de una carta, leerla, cambiarla y cerrarla antes de que llegue.

### 2. Manipulación manual con Repeater

El **Repeater** toma una petición y la reenvía modificada cuantas veces quieras.

```
POST /start-challenge
Content-Type: application/json

{ "challengeId": 3 }
```

Si cambiás un valor y el servidor te deja hacer algo indebido → hallazgo.

### 3. Automatización y fuerza bruta

- **Fuerza bruta:** probar TODAS las combinaciones (PIN de 4 dígitos = 10.000).
- **Diccionario:** probar una LISTA de claves comunes ( `123456` , `password` ...). Más rápido.

**Defensa:** rate limiting, bloqueo de cuenta, captcha, MFA. (Herramienta: Burp **Intruder**.)

## 🟥 Módulo IV — Estandarizar vulnerabilidades y reportar

### 1. CVE y CWE (no confundir)

| | CVE | CWE |
|---|---|---|
| ¿Qué es? | Una vulnerabilidad concreta publicada | Un tipo de debilidad |
| Responde | "¿Qué bug apareció?" | "¿Qué error lo causó?" |
| Formato | `CVE-2021-44228` | `CWE-89` |
| Ejemplo | Log4Shell (Log4j) | CWE-89 = inyección SQL |

### 2. Tríada CIA (los 3 pilares)

| Pilar | Protege | Ataque que lo rompe |
|---|---|---|
| **C**onfidencialidad | Que solo vean quienes deben | IDOR (ver la factura de otro) |

| | | |
|---|---|---|
| **I**ntegridad | Que no se alteren sin permiso | Parameter tampering ( `precio=100` ) |
| **A**vailability | Que el servicio esté disponible | DDoS (saturar el server) |

### 3. Plantilla de reporte de pentesting

Identificación → Resumen ejecutivo → Detalles técnicos → Métricas CVSS → PoC → Recomendaciones → Validación.

### 4. Primer reporte oficial — Turnero (IDOR)

Se pudo **borrar turnos de otro usuario** ( `xdalvik` ) porque el servidor no validaba al dueño. CWE-639/862. **CVSS 8.8 Alta** → `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` .

#### 🎯 Cómo leer un vector CVSS

| Sigla | Significa | Valores |
|---|---|---|
| AV | Desde dónde se ataca | N / A / L / P |
| AC | Complejidad | L / H |
| PR | Permisos necesarios | N / L / H |
| UI | ¿Necesita a la víctima? | N / R |
| S | Alcance | U / C |
| C/I/A | Impacto en cada pilar | N / L / H |

**Lectura rápida:** `AV:N/PR:N/UI:N` + `C:H/I:H/A:H` = por internet, sin credenciales, sin ayuda de la víctima, rompe todo → **crítico (9–10)**.

| | | | | |
|---|---|---|---|---|
| **Puntaje** | 0.1–3.9 Baja | 4.0–6.9 Media | 7.0–8.9 Alta | 9.0–10 Crítica |

## 🟥 Módulo V — Configuraciones inseguras (Security Misconfiguration, A05)

### 1. Caza pasiva de configuraciones inseguras

Observar **sin explotar**: headers faltantes, paneles expuestos, versiones viejas, mensajes que hablan de más. Solo mirar y documentar.

### 2. Ejecución de laboratorios — Reporte Asistencia (VULN-0005)

Lab resuelto y reportado con la plantilla oficial (mala configuración que permitía acceso/manipulación indebida).

### 3. Peligros del Directory Listing

```
Index of /archivos/
backup_2026.zip      12 MB
config.old            3 KB
reporte_clientes.csv  480 KB
```

Si el listado está activo y no hay `index.html`, el server muestra TODOS los archivos. **Defensa:** desactivar el listado y no dejar archivos sensibles en carpetas web.

### 4. Manejo inseguro de errores (Stack Traces)

❌ Inseguro: muestra la traza interna (rutas, framework, versiones). ✅ Seguro: mensaje genérico con un `requestId` para buscar en logs. Un stack trace es un mapa interno regalado al atacante (*Information Disclosure*).

## 🟪 Módulo VI — Autenticación, sesión y JWT

### 1. Auditoría de cookies — banderas de seguridad

| Bandera | Qué hace | Si falta... |
|---|---|---|
| HttpOnly | El JS no puede leer la cookie | Un XSS podría robar la sesión |
| Secure | Solo viaja por HTTPS | Podría ser interceptada por HTTP |
| SameSite | Controla envío entre sitios | Facilita CSRF |

**Matiz:** `HttpOnly` no evita el XSS, solo que roben *esa* cookie. Una cookie que vence al cerrar el navegador no es necesariamente de autenticación.

### 2. Ingeniería inversa a JWT ⭐

```
BASE64URL(Header) . BASE64URL(Payload) . BASE64URL(Signature)
   eyJhbGci...     .     eyJ1c2Vy...     .    FGoloAz...
```

| Parte | Contiene | Ejemplo |
|---|---|---|
| Header | El algoritmo de firma | `{"alg":"HS256"}` |
| Payload | Los datos / ¡quién sos! | `{"user_id":1,"exp":...}` |
| Signature | La firma anti-manipulación | hash con la clave secreta |

> **Base64URL NO es cifrado.** Cualquiera que tenga el token puede **leer** Header y Payload. Lo que no puede (si está bien hecho) es **cambiarlos**, porque la firma no coincidiría. Para ver los valores no hay que "descifrar": pegás el Payload en jwt.io o lo pasás por Base64.

**La vulnerabilidad:** si la `SECRET_KEY` se filtra (o es débil), vos mismo **firmás** un token con el `user_id` del admin. El servidor lo valida con la misma clave y te cree → **escalada a admin**.

### 3. Laboratorios resueltos

- **Consulta de Multas (VULN-0006):** la `SECRET_KEY` se filtraba por un endpoint → JWT de admin firmado → escalada. **CVSS 9.1 Crítica**.
- **Notas Universitarias (VULN-0007):** superusuario por defecto ( `admin/admin` ) + `DEBUG=True` + IDOR en el editor de notas ( `materia_id` / `estudiante_id` editables). **CVSS 9.8 Crítica**.

### 4. Ataque de Algoritmo "none"

```
{ "alg": "none", "typ": "JWT" }
```

Si el servidor acepta `alg:none` , **ignora la firma** y cree cualquier Payload → admin sin conocer la clave. *Analogía:* un control que, si le decís "no me revises", te deja pasar. **Defensa:** exigir el algoritmo correcto y rechazar `none` .

## 🗂️ Taxonomía de ataques — las 9 familias de Software Seguro

| Familia | Pregunta clave | Ejemplos (1 línea c/u) |
|---|---|---|
| **01 Inyecciones** | ¿Separa datos de código? | SQLi ( `admin' --` ) · NoSQLi ( `{"$ne":null}` ) · XSS ( `<img onerror>` ) · SSTI ( `{{7*7}}` →49) · RCE ( `; cat /etc/passwd` ) |
| **02 Control de acceso** | ¿Esto me corresponde? | Broken AC (entrar a `/admin` ) · IDOR ( `/factura/1002` ) · Mass Assignment ( `"rol":"admin"` ) · Lógica de negocio (cupón x20) |
| **03 Autenticación** | ¿Verifica quién soy? | Auth ( `admin/admin` ) · Fuerza bruta · Tokens (JWT) · FIDO · CSRF |
| **04 Servidor y archivos** | ¿Confía de más en rutas/servicios? | File upload ( `shell.php` ) · Path traversal ( `../../etc/passwd` ) · SSRF ( `169.254.169.254` ) · Info. Disclosure · Race conditions · CVE |
| **05 Reversing** | ¿Esconde secretos en el código? | Reversing APK/Desktop · Buffer overflow |
| **06 Cripto y forense** | ¿Cifrado débil? | Criptoanálisis (MD5 sin sal) · Forense (memoria/esteganografía) |
| **07 Tiempo real** | ¿Revalida permisos en vivo? | WebSockets · WebRTC (filtra IP real) |
| **08 IA y LLMs** | ¿Aísla las instrucciones? | Prompt injection ("ignorá tus instrucciones...") |

## 🧰 Caja de herramientas

| Herramienta | Para qué |
|---|---|

| | |
|---|---|
| Burp Suite | Proxy: interceptar, modificar (Repeater), automatizar (Intruder) |
| Nmap | Escanear puertos/servicios (con autorización) |
| Python + requests | Automatizar pedidos (enumerar IDs, probar endpoints) |
| DevTools (F12) | Cookies (Storage), peticiones (Network), JS (Console) |
| jwt.io / Base64 | Decodificar y analizar JWT |
| rockyou.txt | Diccionario de contraseñas |
| CVSS Calculator | Calcular la gravedad de un hallazgo |

## 🗺️ Mapa conceptual — todo el curso de un vistazo




## ✅ Cuadro resumen final

| Familia | La pregunta que la explota | Defensa general |
|---|---|---|
| Inyecciones | ¿Separa mis datos del código? | Parametrizar, escapar la salida, no pasar input a la shell/template |
| Control de acceso | ¿Chequea que esto me corresponde? | Validar permiso y propiedad en el servidor, en cada acción |

| | | |
|---|---|---|
| Autenticación | ¿Verifica bien quién soy? | Claves fuertes, MFA, rate limiting, firmas verificadas, exigir algoritmo |
| Servidor/archivos | ¿Confía de más en rutas/archivos/servicios? | Lista blanca, normalizar rutas, bloquear IPs internas, actualizar |
| Reversing/Cripto | ¿Esconde secretos en código o cifrado débil? | No hardcodear secretos, cripto estándar con sal |
| Tiempo real/IA | ¿Revalida permisos y aísla las instrucciones? | Autorizar cada mensaje, separar instrucción de dato |

> **La madre de todas las preguntas:** ¿el servidor está confiando en algo que controla el usuario sin verificarlo?

---